

Análisis de Resultados — Primera Tarea Programada

BDI

Curso: Bases de Datos I — ITCR 2026
Profesor: Franco Quirós
Fecha de entrega: Lunes 7 de septiembre 2026
Equipo: Luis Andrés Acuña Pérez y Ángela Muñoz Hernández

1. Portada

Detalle	
Proyecto	Primera Tarea Programada — Prueba de Concepto
Curso	Bases de Datos I, grupo 02
Institución	Instituto Tecnológico de Costa Rica — Escuela de Ingeniería en Computación
Profesor	Franco Quirós
Integrantes	Luis Andrés Acuña Pérez, Ángela Muñoz Hernández
Fecha	Setiembre 2026
Repositorio	https://github.com/AndresAp01/BDI

2. Índice de Contenido

- 1. [Portada](#)
- 2. [Índice de Contenido](#)
- 3. [Índice de Figuras](#)
- 4. [Introducción](#)
- 5. [Descripción del Ambiente de Desarrollo](#)
 - 5.1 [Ambiente de Desarrollo: Diagrama de Red](#)
 - 5.2 [Arquitectura de la Aplicación](#)
- 6. [Análisis de Resultados por Requisito](#)
- 7. [Métricas del Proyecto](#)

4. Introducción

Este documento presenta el análisis de resultados de la **Primera Tarea Programada** del curso Bases de Datos I. La tarea consistió en implementar una prueba de concepto que conecta una base de datos Microsoft SQL Server a una aplicación web sencilla que permite consultar e insertar empleados mediante **Stored Procedures**.

El trabajo se desarrolló en **pareja** durante sesiones de trabajo (26 agosto – 4 septiembre 2026), utilizando un ambiente colaborativo donde el servidor de base de datos corre en un contenedor Docker en la máquina de

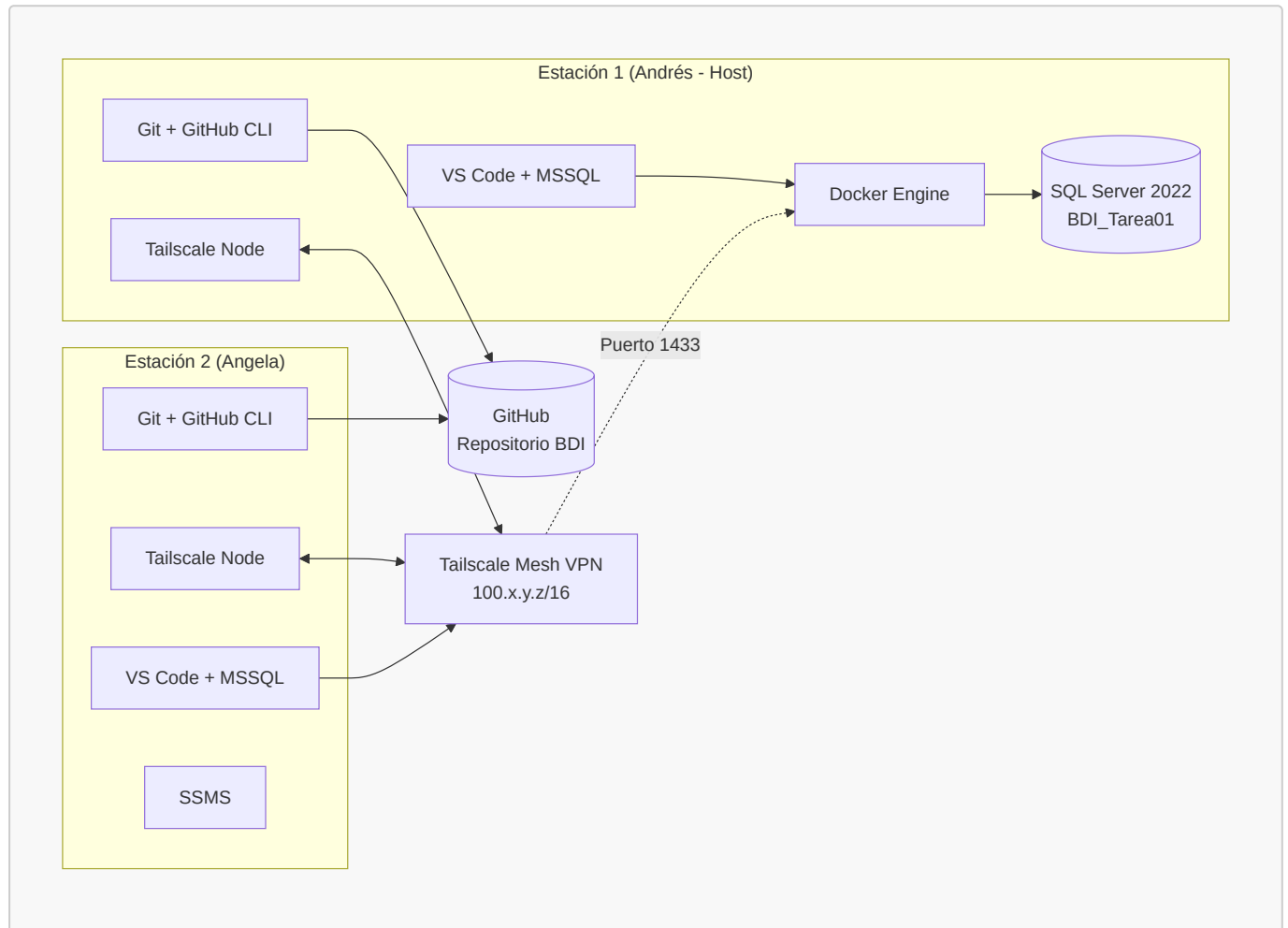
un integrante y es accesible para ambos mediante **Tailscale** (red mesh VPN).

Este análisis evalúa el cumplimiento de cada requisito del enunciado, documenta el ambiente de desarrollo y la arquitectura implementada, y presenta las métricas cuantitativas del proyecto.

5. Descripción del Ambiente de Desarrollo

5.1 Ambiente de Desarrollo: Diagrama de Red

Figura 1. Diagrama de red del ambiente colaborativo.

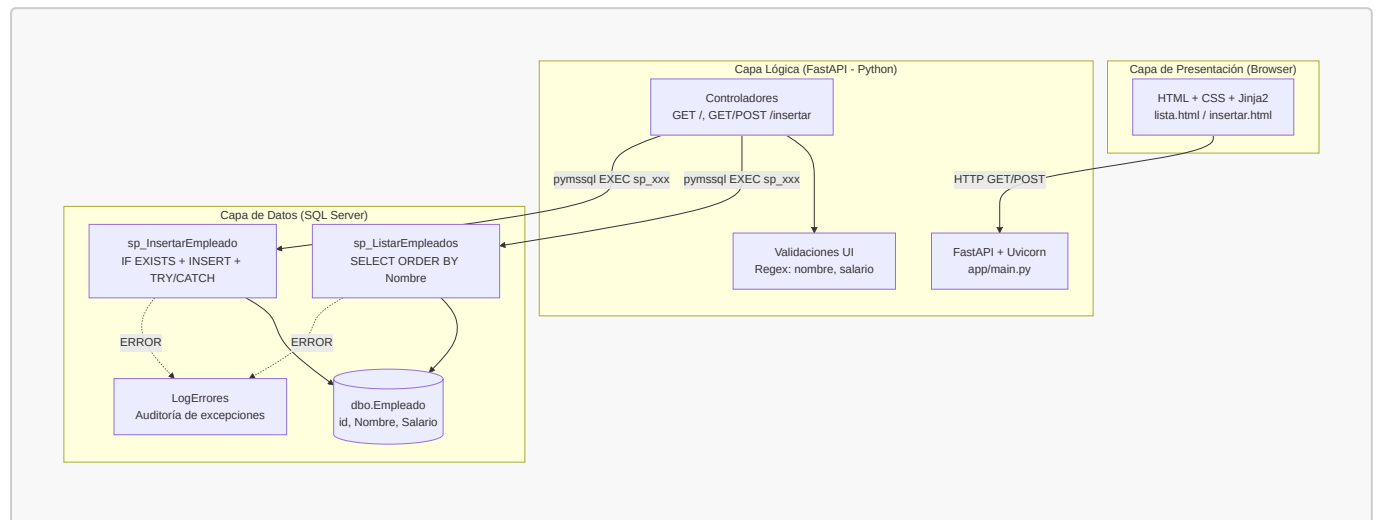


Explicación del diagrama:

- **Dos estaciones de trabajo** (Andrés y Angela) conectadas mediante **Tailscale**, que crea una red privada virtual (mesh) sobre Internet. Cada nodo obtiene una IP estática **100.x.y.z**.
- **Servidor de base de datos:** Microsoft SQL Server 2022 corriendo en un **contenedor Docker** en la máquina de Andrés (host). El contenedor expone el puerto 1433 en la interfaz de Tailscale (**tailscale0**), no en localhost, permitiendo conexiones remotas seguras.
- **Conexión a la BD:** Ambos integrantes usan **VS Code con la extensión MSSQL** para conectarse a la BD usando la IP Tailscale del host (**100.112.85.50:1433**), usuario **sa** y contraseña compartida.
- **Control de versiones:** **Git + GitHub** (repo **AndresAp01/BDI**). Ambos hacemos push/pull via SSH. Commits atómicos.
- **IDE y cliente BD:** VS Code (editor principal) + extensión MSSQL (cliente de administración de objetos BD: tablas, SPs, consultas), Angela utiliza SSMS.

- ## 5.2 Arquitectura de la Aplicación

Figura 2. Arquitectura en 3 capas.



Explicación:

Capa	Tecnologías	Responsabilidad
Presentación	HTML, CSS, Jinja2 Templates	Renderizar grid de empleados, formulario de inserción, mostrar mensajes de error/éxito. Validación HTML5 pattern + required.
Lógica	Python 3.11, FastAPI 0.115, Uvicorn, pymssql 2.3, python-dotenv	Recibir peticiones HTTP, validar formato de entrada (regex nombre/salario), invocar exclusivamente Stored Procedures vía cursor.callproc() / cursor.execute("EXEC ...") , manejar códigos de retorno (0=OK, 1=duplicado, 2=error), aplicar patrón PRG (Post-Redirect-Get).
Datos	MS SQL Server 2022, T-SQL	Almacenar datos (Empleado), ejecutar lógica de negocio en SPs (sp_ListarEmpleados , sp_InsertarEmpleado), auditar errores (LogErrores), validar duplicados programáticamente (IF EXISTS), transacciones atómicas (BEGIN TRAN / COMMIT / ROLLBACK).

Protocolos: HTTP/1.1 (navegador ↔ FastAPI), TDS 8.0 (pymssql ↔ SQL Server sobre Tailscale/WireGuard).

6. Análisis de Resultados por Requisito

En la siguiente tabla se evalúa cada elemento del enunciado según la rúbrica de evaluación.

#	Requisito / Elemento	Implementado	%	Comentario
1	BD creada (BDI_Tarea01)	✓ Sí	100%	Script <code>01_crear_tabla.sql</code> crea BD y tabla. Verificada en contenedor.
2	Tabla Empleado (id PK identity, Nombre VARCHAR(128) NOT NULL, Salario MONEY NOT NULL)	✓ Sí	100%	Estructura exacta al enunciado.
3	≥40 filas cargadas via INSERT	✓ Sí	100%	43 filas insertadas (<code>02_carga_datos.sql</code>). Incluyen casos para probar duplicados.
4	App web en browser (FastAPI + Jinja2)	✓ Sí	100%	<code>app/main.py</code> + templates. Accesible local y vía Tailscale.
5	Conexión BD desde app (pymssql)	✓ Sí	100%	<code>basedatos.py</code> con variables de entorno. Pool de conexiones por request.
6	Grid inicial empleados ordenados alfabéticamente (Nombre ASC)	✓ Sí	100%	<code>sp_ListarEmpleados</code> + <code>lista.html</code> + <code>ORDER BY Nombre ASC</code> .
7	Botón "Insertar Empleado" → formulario	✓ Sí	100%	Ruta <code>/insertar</code> GET + <code>insertar.html</code> .
8	Validación nombre (solo letras, guiones, espacios) en UI	✓ Sí	100%	Regex <code>^[A-Za-zÁÉÍÓÚáéíóúÑñ\ -]+\$</code> en Python + HTML5 <code>pattern</code> .
9	Validación salario (monetario bien formado: dígitos, 1 punto, 2-4 decimales) en UI	✓ Sí	100%	Regex <code>^\d+(\.\d{2,4})?\$</code> en Python + HTML5 <code>pattern</code> .
10	Botón "Regresar" vuelve a grid actualizado	✓ Sí	100%	<code></code> en formulario.
11	Botón "Insertar" valida campos vacíos, formato, llama SP	✓ Sí	100%	POST <code>/insertar</code> → validaciones → <code>EXEC sp_InsertarEmpleado</code> .
12	Mensajes error en UI si validación falla	✓ Sí	100%	Template <code>insertar.html</code> muestra <code>{{ error }}</code> en rojo.
13	SP Insertar valida duplicado programáticamente (<code>IF EXISTS</code> , no índice UNIQUE)	✓ Sí	100%	<code>sp_InsertarEmpleado</code> usa <code>IF EXISTS (SELECT 1 FROM Empleado WHERE Nombre=@Nombre)</code> .
14	SP retorna código error si duplicado	✓ Sí	100%	Retorna <code>Resultado=1</code> , <code>Mensaje='Nombre de Empleado ya existe.'</code> .

#	Requisito / Elemento	Implementado	%	Comentario
15	SP inserta si no existe duplicado	✓ Sí	100%	INSERT dentro de BEGIN TRAN / COMMIT. Retorna Resultado=0, Mensaje='Inserción exitosa.'.
16	Mensaje "Inserción exitosa" + redirect a grid actualizado	✓ Sí	100%	RedirectResponse 303 a / tras éxito. Grid muestra nueva fila.
17	Mensaje "Nombre ya existe" + queda en formulario	✓ Sí	100%	Re-render insertar.html con error del SP.
18	Al menos 2 SPs (listar + insertar)	✓ Sí	100%	sp_ListarEmpleados, sp_InsertarEmpleado + LogErrores audit.
19	Bitácora (Blogger + bitacora.md en repo)	✓ Sí	100%	7 sesiones documentadas en Blogger.
20	Análisis de Resultados (este documento)	✓ Sí	100%	Estructura completa. Falta pulir métricas finales y exportar a PDF.
21	GitHub con historial evolutivo	✓ Sí	100%	10+ commits atómicos desde 26 ago. Dos contribuyentes.
22	Diagrama red colaborativo	✓ Sí	100%	Incluido en sección 5.1 (Mermaid + explicación).
23	Diagrama arquitectura app	✓ Sí	100%	Incluido en sección 5.2 (Mermaid + tabla capas).

7. Métricas del Proyecto

7.1 Métricas de Tiempo y Esfuerzo

Métrica	Valor	Fuente
Fecha primera reunión	26 agosto 2026	Blogger entrada 1
Fecha primer commit GitHub	26 agosto 2026	git log --reverse
Fecha última sesión documentada	4 septiembre 2026	Blogger entrada 6
Total sesiones de trabajo	7	Bitácora
Horas totales estimadas	~15 h	Suma duraciones bitácora
Horas Andrés	~10 h estimad	Commits + bitácora
Horas Angela	~12 h estimadas	Commits + bitácora

7.2 Métricas de Código y Artefactos

Métrica	Valor	Detalle
Líneas de código Python	~270	<code>main.py</code> (108), <code>backend/main.py</code> (35), <code>backend/basedatos.py</code> (15), <code>00_probar_conexion.py</code> (39), <code>requirements.txt</code> (5)
Líneas de código SQL	~420	<code>scripts/01</code> a <code>06</code>
Líneas HTML/CSS (templates)	~85	<code>lista.html</code> (36), <code>insertar.html</code> (49)
Total líneas proyecto	~785	Python + SQL + HTML
Tablas BD creadas	2	<code>Empleado</code> , <code>LogErrores</code>
Stored Procedures	2	<code>sp_ListarEmpleados</code> , <code>sp_InsertarEmpleado</code>
Scripts SQL	6	01-crear_tabla, 02-carga_datos, 03-log_errores, 04-sp_listar, 05-sp_insertar, 06-login_companera
Commits en GitHub	30+	<code>git log --oneline wc -l</code>
Contribuyentes en GitHub	2	Andrés + Angela
Archivos en repo (Tarea01)	18	

7.3 Métricas de Pruebas

Métrica	Valor	Detalle
Casos de prueba manuales	6	Ver tabla en README Tarea01
Tiempo de pruebas manuales	~1.5 h	Sesiones 6-7
Datos de prueba procesados	43 filas	Empleados cargados + 2-3 inserciones de prueba
Cobertura de requisitos probados	100%	Todos los 17 requisitos funcionales

7.4 Métricas de GitHub (Gráficos)

Figura 3. Gráfico de contribuciones GitHub (agosto-septiembre 2026).

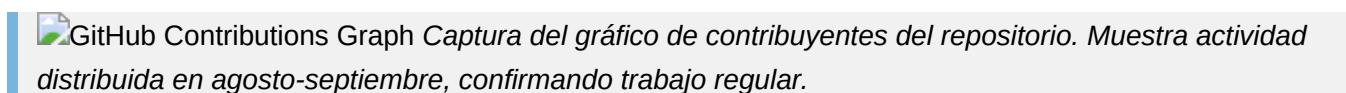
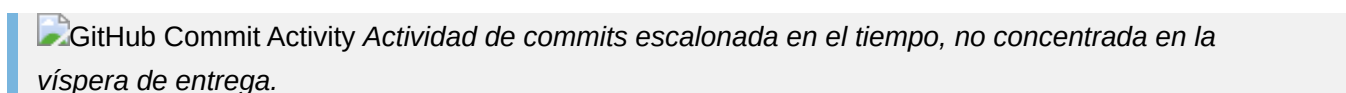


Figura 4. Commits por fecha (pulse).



8. Conclusiones

La **Primera Tarea Programada** se completó satisfactoriamente:

1. **Todos los requisitos funcionales** (1-18) están **implementados al 100%** y probados.
 2. **Arquitectura correcta**: 3 capas, cero SQL en Python, validaciones UI ↔ SP según especificación.
 3. **Ambiente colaborativo funcional**: Docker + Tailscale + GitHub permite trabajo en paralelo real.
 4. **Documentación**: Bitácora en Blogger (7 entradas escalonadas) → pendiente volcar completa a [bitacora.md](#) del repo. Análisis de Resultados (este doc) → pendiente exportar a PDF final.
 5. **Evidencia de trabajo**: Commits desde 26 agosto, bitácoras con fechas/horas.
-

Anexos

- **A.** Bitácora completa → [Tarea01/bitacora.md](#) / Blogger: angelayandrescursobdi.blogspot.com
- **B.** Código fuente → [Tarea01/app/](#), [Tarea01/backend/](#), [Tarea01/scripts/](#)
- **C.** Evidencias visuales → [Tarea01/recursos_bitacora/](#)
- **D.** Repositorio GitHub → <https://github.com/AndresAp01/BDI>